



SIGNAL SERIES • BUILD

PIPELINES, BUILD, TEST & ARTIFACTS

Trigger to build to test to versioned artifact — one flow.

By **Nabawy**

AUTHOR & COMPILER

Pipelines

Artifacts

Table of Contents

03	Pipelines	03	04	Execution	04
05	Stages & Builders	05	06	Discipline	06
07	Handoff	07	08	1. What Is It?	08
09	1. What Is It?	09	10	Testing Strategy · continued	10
11	Artifacts	11	12	Lifecycle	12
13	Trust & Handoff	13	14	 Quick Revision — Terms Table	14
15	 Concept Questions — Distilled Answers	15	16	 Command Cheat Sheet	16
17	 Self-Quiz + Answer Key	17			

■ How to use this book legend

 NOTE · context  KEY · must retain  WATCH OUT · gotcha  VERIFIED · check

NOTE Read cover to cover once, then use the Contents to jump. Every chapter ends with a  EXAM box — if you can answer it cold, move on.

Pipelines

Picking up from *Source Control*, where a merge fired the trigger — we now examine *the pipeline that the trigger starts*: stages of jobs of steps, executed by runners.

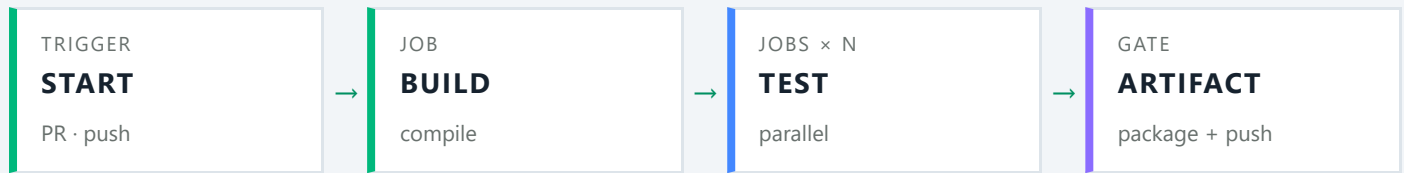
01 / VOCABULARY

Term	Meaning	Example
Pipeline	Whole automated workflow	ci.yml
Stage	Logical phase	build → test → scan
Job	Unit of work (often parallel)	unit-tests
Step	Single operation in a job	npm test
Runner	Machine executing jobs	ubuntu-latest

Threads: [Controllers & Agents](#)

Execution

02 / HOW A RUN EXECUTES



Runner checks out the commit, sets up a cached toolchain, runs build → test → scan (independent jobs in parallel, needs: for gates), and returns results as PR feedback. Fail-fast by default; advisory checks ride continue-on-error.

03 / CANONICAL WORKFLOW

```
# ci.yml – build, then test + scan in parallel
jobs:
  build: # npm ci + build
  test:  needs: build # npm test --coverage
  scan:  needs: build # npm audit + gitleaks
```

Threads: [Controllers & Agents](#)

BUILD VS BUY RUNNERS Hosted runners win when maintenance costs more than minutes — start there. Self-host when compliance, GPUs, or private networks demand it; cache by lockfile either way.

Stages & Builders

04 / THE THREE VERIFICATION STAGES

- **Build** — hermetic: pinned deps, clean env, no hidden host tools. Same commit → byte-identical output anywhere.
- **Test** — unit (fast, per-PR) → integration → E2E/smoke (staging). PR suites stay under 10 min.
- **Security scan** — npm audit, gitleaks, Trivy on every PR. Shifting left beats incident response.

COMPONENT & INTEGRATION PIPELINES multi-binary systems: each component pipeline publishes tested versioned binaries · one integration pipeline assembles them and runs full acceptance. Trigger downstream on fresh upstream binaries; only jointly-green combinations ship.

05 / WITHOUT A DAEMON: BUILDKIT & KANIKO

	BuildKit (buildx)	Kaniko
Needs daemon	Yes (local or remote)	No — userspace build
Best in	Docker agents, dev machines	Locked-down K8s
Secrets	--secret mounts	Short-lived tokens

RULE BuildKit everywhere a daemon exists; Kaniko exactly where daemons are banned.

RUNNERS managed = zero ops, cold caches (use registry/remote cache); self-hosted = warm cache, you own patches + scaling. Cache key must include lockfiles.
DAG — fan out independent jobs, gate fan-in; cap matrix axes; fail-fast for PR signal, advisory for experiments; retry idempotent steps only; quarantine flakes with owner + deadline.

Threads: [Hermetic Builds](#) · [Artifact Digest](#)

Discipline

06 / FAILURE MODES



FLAKY TESTS shared state → quarantine, fix, or delete



SLOW PIPELINE no cache → cache, split, parallelize



WORKS LOCALLY unpinned deps → lockfiles, containerized runners

RULES PR pipeline under 10 min · pin action/tool versions · never bake secrets into logs/images · same steps locally and in CI.

Q1 What belongs in the PR pipeline vs post-merge?

Fast unit/integration + scans in PR; slow E2E and staging gates post-merge.

Threads: [Hermetic Builds](#) · [Supply-Chain Security](#)

Handoff

07 / HANDOFF

This pipeline produces one thing that outlives it — the artifact. Continue in **Artifact Management**, where the versioned output is stored, promoted, and traced.

Build Systems

A build system turns source into runnable output deterministically: dependency resolution, compilation, bundling — the same inputs must always produce the same outputs.

1. WHAT IS IT?

Picking up from **CI/CD Pipelines**, where the build stage was defined — here is what actually runs inside it: Maven/Gradle (JVM), npm (JS), Go modules, Cargo — each resolving declared dependencies into a compiled/bundled artifact.

2. HERMETIC BUILDS

Hermetic = sealed from the ambient machine (see glossary). Concretely: pinned dependency versions (lockfiles: `package-lock.json`, `pom.xml` versions, `go.sum`), clean container per build, no `latest` tags, no host-installed toolchains. Violation symptom: "works in CI, fails locally" in reverse — the build drank something from its environment.

3. CACHING DONE RIGHT

Cache layers, fastest first: dependency cache keyed on lockfiles → remote build cache (Gradle remote, BuildKit registry cache) → artifact reuse (skip rebuild when inputs unchanged). Golden rule: the cache key must include every input; a stale key ships stale code silently.

4. TOOL NOTES

- **Maven/Gradle:** standard layouts, wrapper scripts (`mvnw`, `gradlew`) pin the toolchain itself — commit them.
- **npm:** `npm ci` (lockfile-exact) in CI, never `npm install`; cache `~/.npm`.
- **Docker builds:** BuildKit with `--secret` mounts and registry cache (see pipelines BuildKit section); multi-stage so SDKs never ship in runtime images.

5. FAILURE MODES

- Unpinned deps → "it built yesterday"; fix with lockfiles + dependabot-style updates as PRs.
- Bloated images (SDK in prod) → slow pulls, huge attack surface; fix with multi-stage builds.
- Cache poisoning (stale key) → phantom green; fix by keying on lockfiles + periodic cache-bust verification.

6. INTERVIEW NOTES

- Hermetic vs reproducible: sealed environment vs byte-identical output (reproducible is stricter).
- Why `npm ci` over `npm install` in CI.
- Multi-stage Docker: what ships vs what builds.

RELATED TOPICS

- CI/CD Pipelines
- Artifact Management
- Continuous Integration

Deterministic builds produce trustworthy artifacts. Continue in **Artifact Management**.

Threads: [Hermetic Builds](#) · [Artifact Digest](#) · [Supply-Chain Security](#)

Testing Strategy

Testing strategy decides which tests run where: fast unit tests gate every PR, slow E2E proves staging — the pyramid keeps feedback fast without losing coverage.

1. WHAT IS IT?

Picking up from **Build Systems**, where output became deterministic — now prove it's correct, at the right stage, at the right cost.

2. THE PYRAMID IN PIPELINES

Layer	Speed	Runs in	Guards
Unit	ms each, minutes total	Every PR	Logic correctness
Contract	seconds	Every PR (provider + consumer)	API compatibility between services
Integration	minutes	Post-merge / staging	Wiring, DBs, queues
E2E / smoke	minutes–tens of minutes	Staging, post-deploy	Real user journeys

Many unit, fewer E2E: an inverted pyramid (all Selenium, no units) is slow, flaky, and tells you nothing a unit test couldn't in milliseconds.

Test doubles, cheapest first: stub (canned answers) → fake (working shortcut, e.g. in-memory store) → mock (verifies interactions). Mock roles and boundaries, never implementation trivia — over-mocked suites shatter on every refactor while missing real regressions.

3. PLACEMENT RULES

- PR pipeline: unit + contract + fast integration (< 10 min total). Anything slower moves post-merge.
- Staging: full integration + E2E + smoke after deploy; failures block promotion, not merges.
- Post-deploy: synthetic smoke probes continuously — the pipeline's last test never ends.
- Acceptance criteria as executable specs: analyst + tester + developer agree the criteria before coding, written so the pipeline can run them (Given-When-Then). Docs that can't fail the build drift from behavior.

4. FLAKY POLICY

Flaky test protocol: quarantine to a non-blocking suite immediately (same day), assign an owner, fix-or-delete deadline of one sprint. A flaky gate teaches developers that red is meaningless — the most expensive test debt there is.

Threads: [Artifact Digest](#)

Testing Strategy · continued

5. CAPACITY & PERFORMANCE PLACEMENT

Functional tests prove correctness; capacity tests prove survival. Treat performance as a requirement with a number, not a wish:

- Quantify first: convert expected load into thresholds (p99 latency, sustained throughput, concurrent users) — unquantified goals justify unbounded effort.
- Placement: short performance smoke sentinels in the PR pipeline; full load scenarios post-merge/staging on isolated, production-like hosts (shared or noisy hosts produce phantom regressions).
- Derive load from reality: adapt acceptance flows into composable load scenarios; warm up caches/JIT before measuring.
- Ratchet thresholds: start at minimum viable target, raise as headroom proves stable; investigate before lowering.
- Vocabulary: latency (one transaction's time) vs throughput (transactions per interval) vs capacity (max sustainable throughput within latency budget).

6. FAILURE MODES

- Testing pyramid inverted → 40-minute PR suites developers ignore; rebalance toward units.
- E2E in PR → queue pileups; move to staging.
- No contract tests in microservices → integration surprises at deploy; add consumer-driven contracts.
- Coverage % as a target → gaming (assert-free tests); treat coverage as smoke signal, not goal.

7. INTERVIEW NOTES

- Why contract tests exist (microservice API drift).
- Flaky quarantine protocol, step by step.
- What runs in PR vs staging vs post-deploy, and why.

RELATED TOPICS

- CI/CD Pipelines
- Build Systems
- Continuous Delivery

Proven correct — now ship it releasably. Continue in **Continuous Delivery**.

🎓 **EXAM** Pipelines in one breath: a merge fires stages of jobs of steps on runners; build **hermetic** (pinned lockfiles, clean env), test by **pyramid** (fast units per PR, slow E2E post-merge, flakes quarantined same-day), scan every PR; **BuildKit** where daemons exist, **Kaniko** where banned; PR suites stay under 10 min.

➡ **Next: Artifacts**

Threads: **Hermetic Builds**

Artifacts

Picking up from *Pipelines*, where the build succeeded — we now follow *the one thing that outlives the pipeline*: the versioned, immutable artifact.

01 / VOCABULARY

Term	Meaning
Artifact	Built output + version (app:1.4.2, app:sha-9f3a)
Registry	Storage + distribution (Hub, GHCR, ECR, Nexus)
Promotion	Same artifact approved staging → production (never rebuild)
Digest	Content hash — the image's fingerprint

02 / WHY IMMUTABILITY

Rebuilding per environment means testing one binary and shipping another. Immutable artifacts guarantee **what passed staging is what runs in production**.

Threads: [Artifact Digest](#)

Lifecycle

03 / PRODUCER, CONSUMER, LIFECYCLE

PRODUCER build job → **STORAGE** registry → **CONSUMER** CD pipeline · Lifecycle: build → store → promote → deploy → retire.

04 / COMMANDS

```
$ docker build -t ghcr.io/acme/app:sha-9f3a .  
$ docker push ghcr.io/acme/app:sha-9f3a  
$ docker tag ghcr.io/acme/app:sha-9f3a ghcr.io/acme/app:1.4.2 # promote: same digest, human tag  
$ docker push ghcr.io/acme/app:1.4.2
```

05 / VERSIONING

- Immutable primary tag: commit SHA — traceable, never overwritten.
- Human tags: semver (MAJOR.MINOR.PATCH) applied at promotion, never rebuilt.
- Never deploy :latest — it is unrepeatable.
- Also valid: build numbers (build-1042) for ordering, date tags for nightlies. SHA always = truth.

Threads: [Artifact Digest](#)

Trust & Handoff

06 / FAILURE MODES

- Mutable tags overwritten → unreproducible deploys; fix with SHA digests + tag immutability.
- Registry bloat/slow pulls; fix with retention + layer caching + regional mirrors.
- Secrets baked into images; fix with multi-stage builds + secret mounts, scan with Trivy.

SUPPLY-CHAIN TRUST

Sign artifacts (cosign proves who built the image) and keep SBOMs (the machine-readable ingredient list) so a future CVE traces to exactly the affected images. Provenance attestations (SLSA-style) let consumers verify builder + commit before promoting.

Q2 Why must the same digest flow through all environments?

Otherwise you tested one binary and shipped another — verification is void.

07 / HANDOFF

The artifact is frozen — now it must travel. Continue in **Continuous Delivery**, where this exact digest moves through staging to production.

🎓 **EXAM** Artifacts in one breath: the build publishes one **immutable SHA digest** to the registry; staging and production deploy the **same digest**, promotion only adds human tags (never **:latest**, never rebuild); sign with **cosign**, keep **SBOMs** and provenance — what passed staging is what runs in production.

Threads: [Artifact Digest](#) · [Supply-Chain Security](#)



Quick Revision — Terms Table

Pipeline	Whole automated workflow from trigger to artifact (ci.yml).
Stage	Logical phase: build, test, scan.
Job	Unit of work inside a stage, often parallel.
Step	Single operation inside a job (npm test).
Runner	Machine executing jobs (ubuntu-latest).
Hermetic build	Sealed from the host: pinned deps, clean env, no latest tags.
Lockfile	Pinned dependency manifest (package-lock.json, go.sum).
BuildKit	Daemon-based builder; --secret mounts, registry cache.
Kaniko	Daemonless userspace image builder for locked-down clusters.

Test pyramid	Many fast units per PR, fewer slow E2E in staging.
Contract test	Guards API compatibility between services.
Flaky test	Non-deterministic test; quarantine same-day, fix or delete.
Artifact	Built output plus version (app:sha-9f3a).
Registry	Artifact storage and distribution (GHCR, ECR, Nexus).
Digest	Content hash fingerprinting the exact image.
Promotion	Approving the same digest onward; never rebuild.
Semver	Human MAJOR.MINOR.PATCH tags applied at promotion.
SBOM	Machine-readable ingredient list tracing CVEs to images.

KEY IDEA Cover the right column and define each term aloud. Any stumble goes back to its chapter.

Threads: [Hermetic Builds](#) · [Artifact Digest](#) · [Controllers & Agents](#)



Concept Questions — Distilled Answers

Q1 What belongs in the PR pipeline vs post-merge?

Fast unit, contract, integration plus scans in PR; slow E2E and staging gates post-merge.

Q2 Why must the same digest flow through every environment?

Otherwise you tested one binary and shipped another — verification is void.

Q3 Hermetic vs reproducible?

Hermetic seals the environment; reproducible additionally guarantees byte-identical output.

Q4 Why npm ci over npm install in CI?

ci installs lockfile-exact; install can drift to newer versions.

Q5 BuildKit vs Kaniko rule?

BuildKit everywhere a daemon exists; Kaniko exactly where daemons are banned.

Q6 Flaky test protocol, step by step?

Quarantine to a non-blocking suite same-day, assign an owner, fix-or-delete within one sprint.

Q7 Why do contract tests exist?

Consumer-driven checks catch microservice API drift before deploy.

Q8 Multi-stage Docker: what ships vs what builds?

SDKs and toolchains build in early stages; only the runtime lands in the final image.

Threads: [Hermetic Builds](#) · [Artifact Digest](#)



Command Cheat Sheet

Docker

<code>docker build -t ghcr.io/acme/app:sha-9f3a .</code>	versioned image from SHA
<code>docker push ghcr.io/acme/app:sha-9f3a</code>	publish immutable digest
<code>docker tag ghcr.io/acme/app:sha-9f3a ghcr.io/acme/app:1.4.2</code>	promote: same digest, human tag

JS build & verify

<code>npm ci</code>	lockfile-exact install for CI
<code>npm test --coverage</code>	gated unit suite after build
<code>npm audit + gitleaks</code>	dependency and secret scan per PR

```
jobs:
  build: # npm ci + build
  test:  needs: build # npm test --coverage
  scan:  needs: build # npm audit + gitleaks
```

NOTE Diagnostic order: red gate first — read the failing check, reproduce locally, fix, re-verify. Never bypass.

Threads: [Artifact Digest](#)

Self-Quiz + Answer Key

Q1 Name the pipeline hierarchy top to bottom.

Q2 What makes a build hermetic?

Q3 When is Kaniko the right builder?

Q4 What runs in the PR pipeline?

Q5 First action on a flaky gate?

Q6 What primarily tags an artifact?

Q7 Why never deploy :latest?

Q8 What proves who built an image?

Answer key

1. Pipeline, stage, job, step.
2. Pinned lockfiles, clean container, no latest tags or host toolchains.
3. Where daemons are banned, e.g. locked-down Kubernetes.
4. Unit, contract, fast integration plus scans — under 10 minutes.
5. Quarantine it same-day so red stays meaningful.
6. Immutable commit SHA; semver is added at promotion.
7. It is unrepeatable — you cannot trace or reproduce it.
8. cosign signature plus SBOM and provenance attestation.

FINAL CHECK Eight for eight from memory, twice in a row — the build arc is yours. Open **Delivery, Deployment & Operations** next.

Threads: [Artifact Digest](#) · [Hermetic Builds](#) · [Supply-Chain Security](#)